

IMPLEMENTACIÓN Y ANÁLISIS DE ALGORITMOS DE BÚSQUEDA Y ORDENAMIENTO EN JAVA

Andrea Álvarez

Bootcamp Java Developer



Objetivos del proyecto

01

Implementar y analizar
cinco algoritmos de
búsqueda y
ordenamiento en Java
con el fin de comprender
su funcionamiento

02

Realizar pruebas
cronometradas para
probar la eficiencia de
cada algoritmo con un
conjunto de datos
pequeños, mediano y
grande.

03

Presentar los resultados
en gráficas ilustrativas y
tablas resúmenes para
facilitar su análisis y
evidenciar sus
diferencias de
rendimiento.

01

Algoritmos Implementados

Explicación del funcionamiento y el código
implementado





1. Búsqueda Binaria

ALGORITMO DE BÚSQUEDA

- Requiere que la lista o el arreglo de elementos esté ordenado

Compara el valor buscado con el elemento del medio, **si este es igual termina el proceso**. Si el valor es menor, busca en la mitad izquierda y si es mayor en la mitad derecha. Este se repite dividiendo el espacio de búsqueda a la mitad cada vez

{((({>>}))<<}

```
//recibe un arreglo ordenado, el índice inicial (x), el índice final (sizeArreglo) y el número a buscar
public int busquedaBinaria(int arreglo[], int x, int sizeArreglo, int numero) {

    while (x <= sizeArreglo) {
        // Calcula el índice medio
        int indice = (x + sizeArreglo) / 2;
        // Verifica si el número está en el índice medio
        if (arreglo[indice] == numero) {
            return indice; // Retorna el índice del número encontrado
        } else if (arreglo[indice] > numero) {
            sizeArreglo = indice - 1; // Busca en la mitad izquierda
        } else {
            x = indice + 1; // Busca en la mitad derecha
        }
    }
    return -1; // Retorna -1 si el número no se encuentra en el arreglo
}
```

	Mejor Caso	Peor Caso	Caso Promedio
Tiempo	O(1)	O(log n)	O(log n)



2. Búsqueda Secuencial

ALGORITMO DE BÚSQUEDA



[]

Recorre una lista o arreglo de elementos y **va comparando cada elemento con el valor buscado.**

- Si lo encuentra, este puede **devolver la posición o el dato según se programe.**
- Si llega hasta el final sin encontrarlo, **indicará que no se encuentra entre los elementos.**

{ }

{((({>>}))<<)}

```
public static int busquedaSecuencial(int arreglo[], int numero) {  
    // Verifica si el arreglo es nulo o tiene longitud cero  
    int n = arreglo.length;  
  
    for (int i = 0; i < n; i++) {  
        // Si el elemento en la posición i es igual al número buscado, retorna la posición  
        if (arreglo[i] == numero)  
            return i;  
    }  
    return -1; // Retorna -1 si el número no se encuentra en el arreglo  
}
```

	Mejor Caso	Peor Caso	Caso Promedio
Tiempo	$O(1)$	$O(n)$	$O(n)$

[]



1. Ordenamiento Burbuja

ALGORITMO DE ORDENAMIENTO



{ }

Compara dos elementos que estén a la par y los intercambia si están ubicados en el orden incorrecto. Al terminar cada ronda, **el elemento más grande pasa al final del arreglo**. Este proceso se repite hasta que ya no se puedan hacer intercambio

{ }

	Mejor Caso	Peor Caso
Tiempo	$O(n)$	$O(n^2)$

{ ((({ >> })) <<) }

```
public static void ordenarBurbuja(int arreglo[], int sizeArreglo) {  
  
    boolean intercambio;  
    int auxiliar;  
  
    for (int i = 0; i < sizeArreglo - 1; i++) {  
        intercambio = false; // Inicializa la variable de intercambio como falsa  
        for (int j = 0; j < sizeArreglo - i - 1; j++) {  
            //Compara los elementos del arreglo  
            if (arreglo[j] > arreglo[j + 1]) {  
  
                //Intercambia los elementos si están en el orden incorrecto  
                auxiliar = arreglo[j]; //Guarda el valor del elemento actual  
                arreglo[j] = arreglo[j + 1]; // Asigna el valor del siguiente elemento al actual  
                arreglo[j + 1] = auxiliar; //Asigna el valor guardado al siguiente elemento  
                intercambio = true; //Marca que se ha realizado un intercambio  
            }  
        }  
        if (intercambio == false) { //Si no se realizaron intercambios, el arreglo ya está  
            ordenado por lo que se puede salir del bucle  
            break;  
        }  
    }  
}
```

[]



2. Ordenamiento por inserción

ALGORITMO DE ORDENAMIENTO



[]

Toma cada elemento del conjunto y **lo inserta en la posición correcta** dentro de la parte **ya ordenada**.

Tiempo	Mejor Caso	Peor Caso	Caso Promedio
	$O(n)$	$O(n^2)$	$O(n^2)$

{ }

{ { { { { >> } } } } << }



```
public static void ordenamientoInsercion(int arr[], int arregloSize) {  
  
    for (int i = 1; i < arregloSize; ++i) {  
        int indice = arr[i];  
        int posicion = i - 1;  
  
        // Mueve los elementos del arreglo que son mayores que el indice a una posición adelante  
        while (posicion >= 0 && arr[posicion] > indice) {  
            // Mueve el elemento a la derecha  
            arr[posicion + 1] = arr[posicion];  
            posicion = posicion - 1; // Disminuye la posición  
        }  
        arr[posicion + 1] = indice; // Inserta el indice en la posición correcta  
    }  
}
```

{ }



3. Ordenamiento por selección

ALGORITMO DE ORDENAMIENTO



Busca el **mínimo elemento** y lo **coloca en la primera posición**, continua con **siguientes mínimos** y **lo coloca en la segunda posición**, así continúa este ciclo hasta ordenarlo todo.



Tiempo	Siempre
	$O(n^2)$

{((({>>}))<<}

```
public static void OrdenamientoSeleccion(int[] arreglo, int sizeArreglo) {
    for (int i = 0; i < sizeArreglo - 1; i++) {
        int minimo_indice = i; //Inicializa el índice del elemento mínimo como el índice actual

        // Encuentra el índice del elemento mínimo en el subarreglo no ordenado
        for (int j = i + 1; j < sizeArreglo; j++) {
            if (arreglo[j] < arreglo[minimo_indice]) { //Compara el elemento actual con el mínimo encontrado
                minimo_indice = j; //Actualiza el índice del mínimo si se encuentra un elemento menor
            }
        }

        int auxiliar = arreglo[i]; // Guarda el valor del elemento actual
        arreglo[i] = arreglo[minimo_indice]; // Asigna el valor del elemento mínimo al elemento actual
        arreglo[minimo_indice] = auxiliar; // Asigna el valor guardado al elemento mínimo encontrado
    }
}
```





02 Pruebas de rendimiento

Resultados en segundos y análisis a partir de estos



Método System.nanoTime()

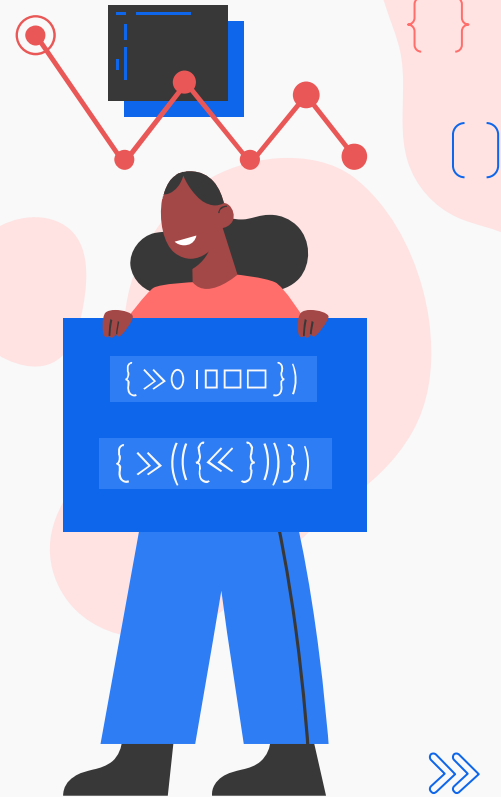
Para realizar estas pruebas utilice el método `System.nanoTime()`, cree dos variables, **inicio y final**, para medir el tiempo de ejecución justo antes de llamar a la clase que contiene al algoritmo hasta el final de la ejecución, así como se muestra a continuación:

```
//Implementacion de algoritmo de ordenamiento por seleccion
OrdenamientoPorSeleccion ordenSeleccion = new OrdenamientoPorSeleccion();

long inicio = System.nanoTime();
ordenSeleccion.OrdenamientoSeleccion(arregloDesordenadoBig, sizeArreglo);

System.out.print("\nArreglo ordenado con el algoritmo de ordenamiento por seleccion: ");
for (int i = 0; i < sizeArreglo; i++) {
    System.out.print(arregloDesordenadoBig[i] + " ");
}

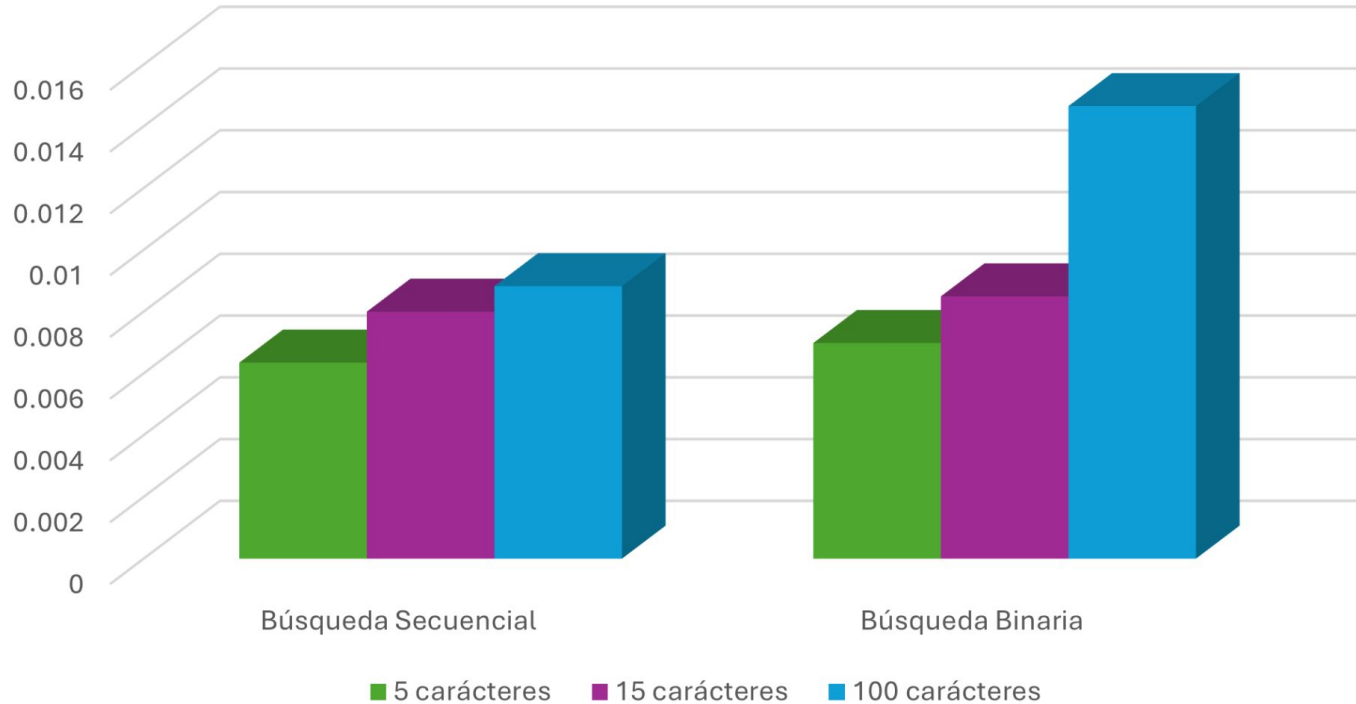
long fin = System.nanoTime();
System.out.println("\nTiempo de ejecución del algoritmo: " + (fin - inicio) + " nanosegundos");
```



Algoritmos de Búsqueda

	Conjunto de datos de prueba		
	Pequeños (5 caracteres)	Medianos (15 caracteres)	Grandes (100 caracteres)
Búsqueda Secuencial	0.0063472 s	0.0079981 s	0.0088221 s
Búsqueda Binaria	0.006982 s	0.0084929 s	0.0146531 s

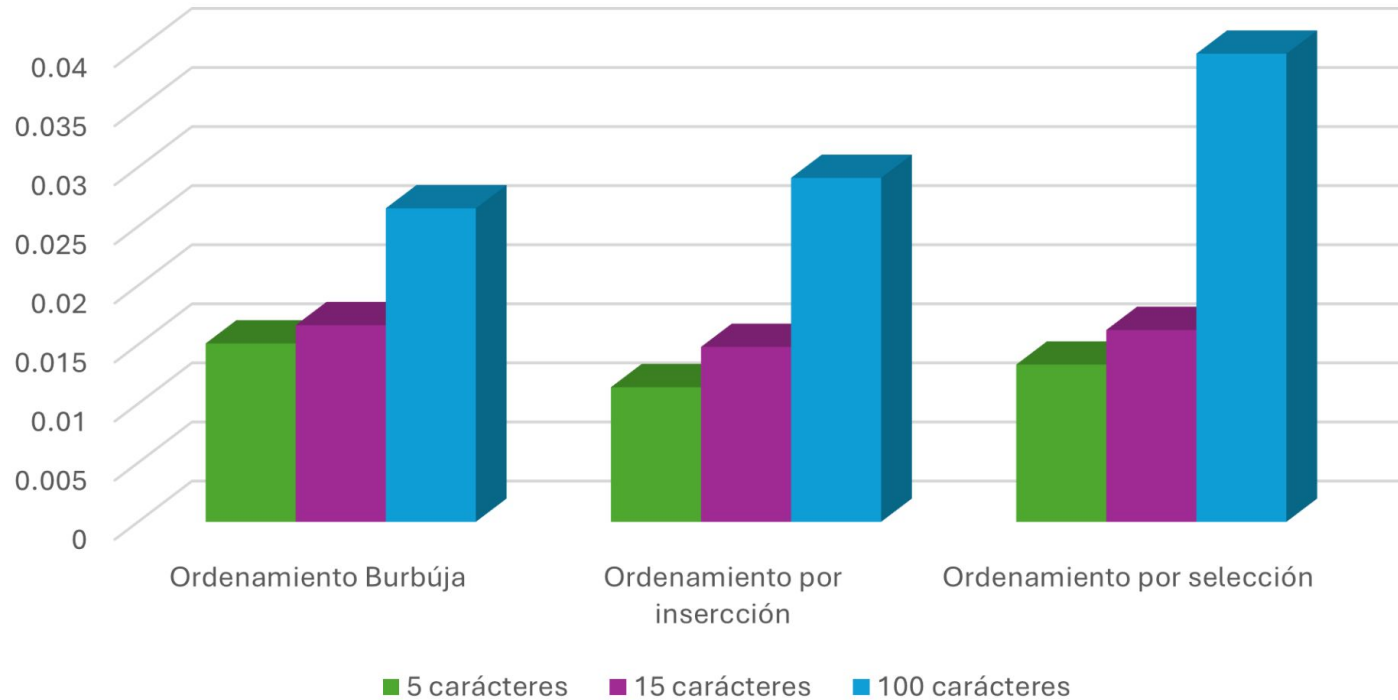
Algoritmos de Búsqueda



Algoritmos de Ordenamiento

	Conjunto de datos de prueba		
	Pequeños (5 caracteres)	Medianos (15 caracteres)	Grandes (100 caracteres)
Ordenamiento burbuja	0.0150971 s	0.0166448 s	0.0265351 s
Ordenamiento por inserción	0.0113939 s	0.0148028 s	0.0291104 s
Ordenamiento por selección	0.0133069 s	0.0162366 s	0.039618 s

Algoritmos de Ordenamiento





Algoritmos de búsqueda



Más recomendado para
conjunto de **listas con**
elementos pequeños y cuando
estén desordenadas.

Secuencial



No es recomendado para
listas grandes por su gran
tiempo de proceso

- Listas ordenadas
previamente



Binaria

Eficiente con **listas grandes y cuando**
se realizan muchas búsquedas.





Algoritmos de Ordenamiento



Útil para **listas pequeñas**
o que casi están
ordenada

Inserción



Burbuja

Solamente es eficiente con **listas pequeñas.**

Usados para fines didácticos



Selección

Siempre tiene el mismo
rendimiento. Es práctico cuando los
intercambios son costosos pero las comparaciones no





¡Gracias por su atención!



Andrea Álvarez

Bootcamp Java Developer

